ĐẠI HỌC BÁCH KHOA HÀ NỘI
TRƯỜNG CÔNG NGHỆ THÔNG TIN VÀ TRUYỀN THÔNG

# BÁO CÁO BÀI TẬP LỚN

## MÔN HỌC: NHẬP MÔN AN TOÀN THÔNG TIN

**Chủ đề:** ***Xác thực Kerberos***

**Giảng viên hướng dẫn:** **PGS.TS. Nguyễn Linh Giang**

**Sinh viên thực hiện:** **Nhóm 4**

1. Nguyễn Phong MSSV: 202400066
2. Nguyễn Ngọc Tuấn Anh MSSV: 202400029
3. Nguyễn Tấn Dũng MSSV: 20230021

**Hà Nội - 2026**

# PHÂN CÔNG THÀNH VIÊN TRONG NHÓM

| STT | Họ và tên | MSSV | Email | Công việc thực hiện | Tỷ lệ |
|---|---|---|---|---|---|
| 1 | Nguyễn Phong | 202400066 | phong.n2400066@sis.hust.edu.vn | Tìm hiểu tổng quan Kerberos, Cơ sở mật mã học, Viết code mô phỏng Client | 33.3% |
| 2 | Nguyễn Ngọc Tuấn Anh | 202400029 | anh.nnt2400029@sis.hust.edu.vn | Vẽ sơ đồ hệ thống, Phân tích kiến trúc 6 bước, Viết code mô phỏng KDC/Server | 33.3% |
| 3 | Nguyễn Tấn Dũng | 20230021 | dung.nt230021@sis.hust.edu.vn | Phân tích các dạng tấn công an toàn thông tin (PtT, Golden Ticket), Tổng hợp báo cáo | 33.3% |

# MỞ ĐẦU

Trong các hoạt động giao tiếp trên mạng máy tính và đặc biệt là môi trường doanh nghiệp hiện đại, việc đảm bảo danh tính của người dùng và các dịch vụ là một trong những yêu cầu cốt lõi của An toàn thông tin. Các phương thức xác thực truyền thống bằng mật khẩu gửi qua mạng dạng rõ tiềm ẩn rất nhiều rủi ro như tấn công nghe lén (sniffing) hay tấn công phát lại (replay attack).

Điều này nói lên sự cần thiết của một cơ chế xác thực an toàn, mạnh mẽ và xuyên suốt. Hiện nay, giao thức Kerberos được xem là tiêu chuẩn vàng trong việc xác thực mạng máy tính (Network Authentication). Được tích hợp chặt chẽ vào Microsoft Active Directory và nhiều hệ thống Unix/Linux, Kerberos cung cấp cơ chế Đăng nhập một lần (Single Sign-On - SSO) an toàn dựa trên mật mã khóa đối xứng.

Tuy nhiên, mức độ an toàn của Kerberos đến đâu và cơ chế hoạt động của nó diễn ra như thế nào để vừa đảm bảo tính bảo mật, vừa đem lại sự tiện lợi cho người dùng? Trên cơ sở đánh giá mức độ an toàn của các hệ thống xác thực, nhóm chúng em đã chọn đề tài: **"Tìm hiểu về Xác thực Kerberos"** làm đối tượng để nghiên cứu phục vụ cho Bài tập lớn môn học.

Do khả năng còn hạn chế và thời gian tìm hiểu có hạn nên mặc dù nhóm đã rất cố gắng hoàn thành tốt nhất nhiệm vụ của mình nhưng không tránh khỏi còn có nhiều thiếu sót. Nhóm em rất mong nhận được những nhận xét thẳng thắn, chi tiết đến từ thầy để tiếp tục hoàn thiện hơn nữa.

Cuối cùng, nhóm xin được gửi lời cảm ơn đến thầy PGS.TS. Nguyễn Linh Giang đã hướng dẫn nhóm trong suốt quá trình hoàn thiện Bài tập lớn. Xin chân thành cảm ơn thầy.

# NHẬN XÉT CỦA GIẢNG VIÊN HƯỚNG DẪN

.....

.....

.....

.....

.....

.....

.....

.....

# Mục lục


**PHÂN CÔNG THÀNH VIÊN** 1

**MỞ ĐẦU** 2

**NHẬN XÉT** 3

**1 TỔNG QUAN VỀ XÁC THỰC VÀ GIAO THỨC KERBEROS** 6
- 1.1 Thực trạng các phương pháp xác thực truyền thống . . . 6
- 1.2 Lịch sử hình thành và phát triển của Kerberos . . . 6
- 1.3 Các khái niệm cốt lõi trong hệ thống Kerberos . . . 7
  - 1.3.1 Principal (Thực thể) . . . 8
  - 1.3.2 Key Distribution Center (KDC - Trung tâm phân phối khóa) . . . 8
  - 1.3.3 Ticket (Vé xác thực) . . . 9
  - 1.3.4 Session Key (Khóa phiên) và Authenticator (Bộ xác thực) . . . 9

**2 CƠ SỞ LÝ THUYẾT MẬT MÃ TRONG KERBEROS** 10
- 2.1 Mật mã khóa đối xứng (Symmetric-Key Cryptography) . . . 10
  - 2.1.1 Các thuật toán mã hóa được Kerberos sử dụng . . . 10
- 2.2 Khóa dài hạn (Long-term Key) và Khóa phiên (Session Key) . . . 11
- 2.3 Bảo vệ tính toàn vẹn (Integrity) bằng MAC và HMAC . . . 11
- 2.4 Nền tảng giao thức Needham-Schroeder . . . 12

**3 KIẾN TRÚC VÀ CƠ CHẾ HOẠT ĐỘNG CHI TIẾT** 12
- 3.1 Pha 1: Dịch vụ xác thực ban đầu (Authentication Service Exchange) . . . 13
  - 3.1.1 Bước 1: AS-REQ (Authentication Service Request) . . . 13
  - 3.1.2 Bước 2: AS-REP (Authentication Service Reply) . . . 14
- 3.2 Pha 2: Dịch vụ cấp vé (Ticket Granting Service Exchange) . . . 14
  - 3.2.1 Bước 3: TGS-REQ (Ticket Granting Service Request) . . . 14
  - 3.2.2 Bước 4: TGS-REP (Ticket Granting Service Reply) . . . 15
- 3.3 Pha 3: Trao đổi giữa Client và Server (Client/Server Exchange) . . . 15
  - 3.3.1 Bước 5: AP-REQ (Application Request) . . . 15
  - 3.3.2 Bước 6: AP-REP (Application Reply - Tùy chọn nhưng quan trọng) . . . 16
- 3.4 Xác thực liên miền (Cross-Realm Authentication) . . . 16

**4 PHÂN TÍCH AN TOÀN VÀ CÁC DẠNG TẤN CÔNG TRONG KERBEROS** 16
- 4.1 Mô hình hóa rủi ro và các điểm yếu nội tại . . . 16
- 4.2 Tấn công Pass-the-Ticket (PtT) . . . 17
- 4.3 Tấn công Kerberoasting . . . 18
- 4.4 Tấn công Golden Ticket và Silver Ticket . . . 18
  - 4.4.1 Golden Ticket (Vé Vàng) . . . 19
  - 4.4.2 Silver Ticket (Vé Bạc) . . . 19
- 4.5 Đánh giá lại Tấn công phát lại (Replay Attack) . . . 19

**5 CHƯƠNG TRÌNH THỬ NGHIỆM VÀ MÔ PHỎNG** **19**
5.1 Môi trường và Công cụ thử nghiệm . . . . . 20
5.2 Thiết kế và Cấu trúc mã nguồn . . . . . 20
5.3 Cảnh báo học thuật và Phạm vi mô phỏng . . . . . 20
5.4 Phân tích Kịch bản khởi chạy và Cấu trúc hệ thống (`main.cpp`) . . . . . 21
5.5 Phân tích Luồng xác thực từ góc nhìn Client (`kerberos_client.cpp`) . . . . . 21
5.5.1 Pha 1: AS Exchange (Xin cấp TGT) . . . . . 22
5.5.2 Pha 2: TGS Exchange (Xin cấp Service Ticket) . . . . . 22
5.5.3 Pha 3: Application Exchange (Xác thực hai chiều) . . . . . 23
5.6 Kết luận thử nghiệm . . . . . 23

**KẾT LUẬN CHUNG** **24**

**TÀI LIỆU THAM KHẢO** **25**

# 1 TỔNG QUAN VỀ XÁC THỰC VÀ GIAO THỨC KERBEROS

## 1.1 Thực trạng các phương pháp xác thực truyền thống

Trong môi trường mạng máy tính sơ khai, việc xác thực người dùng thường được thực hiện thông qua các cơ chế đơn giản như Basic Authentication hoặc các phương thức truyền mật khẩu dưới dạng văn bản rõ (plaintext). Mặc dù dễ triển khai, các phương pháp này bộc lộ những lỗ hổng bảo mật chết người khi áp dụng vào các hệ thống mạng quy mô lớn hoặc mạng công cộng không an toàn.

Một kịch bản điển hình của xác thực truyền thống là khi Client gửi trực tiếp thông tin đăng nhập (tên người dùng và mật khẩu) đến Server. Trong một mạng cục bộ (LAN) không được phân mảnh tốt hoặc trên môi trường Internet, một kẻ tấn công (Attacker) có thể dễ dàng sử dụng các công cụ đánh hơi mạng (Network Sniffing) như Wireshark hoặc tcpdump để bắt các gói tin này. Vì dữ liệu không được mã hóa, mật khẩu của người dùng sẽ bị lộ ngay lập tức.

Hơn nữa, ngay cả khi mật khẩu được băm (hash) trước khi gửi qua mạng (như giao thức NTLM cũ của Microsoft), hệ thống vẫn phải đối mặt với các rủi ro nguy hiểm khác:

- **Tấn công phát lại (Replay Attack):** Kẻ tấn công không cần biết mật khẩu thật, chúng chỉ cần bắt lại gói tin chứa mã băm hợp lệ và gửi lại (phát lại) cho Server vào một thời điểm khác để chiếm quyền truy cập.

- **Tấn công giả mạo máy chủ (Spoofing/Man-in-the-Middle):** Client gửi thông tin đăng nhập cho Server nhưng không có cơ chế nào để Client xác minh xem Server đó có thực sự là máy chủ hợp pháp hay không. Kẻ tấn công có thể dựng lên một máy chủ giả mạo để lừa Client giao nộp thông tin.

- **Khó khăn trong quản lý diện rộng:** Khi số lượng Server tăng lên, việc người dùng phải nhớ và nhập mật khẩu nhiều lần cho từng dịch vụ khác nhau gây ra sự bất tiện lớn, đồng thời làm tăng nguy cơ lộ lọt do người dùng có xu hướng đặt mật khẩu yếu hoặc ghi ra giấy.

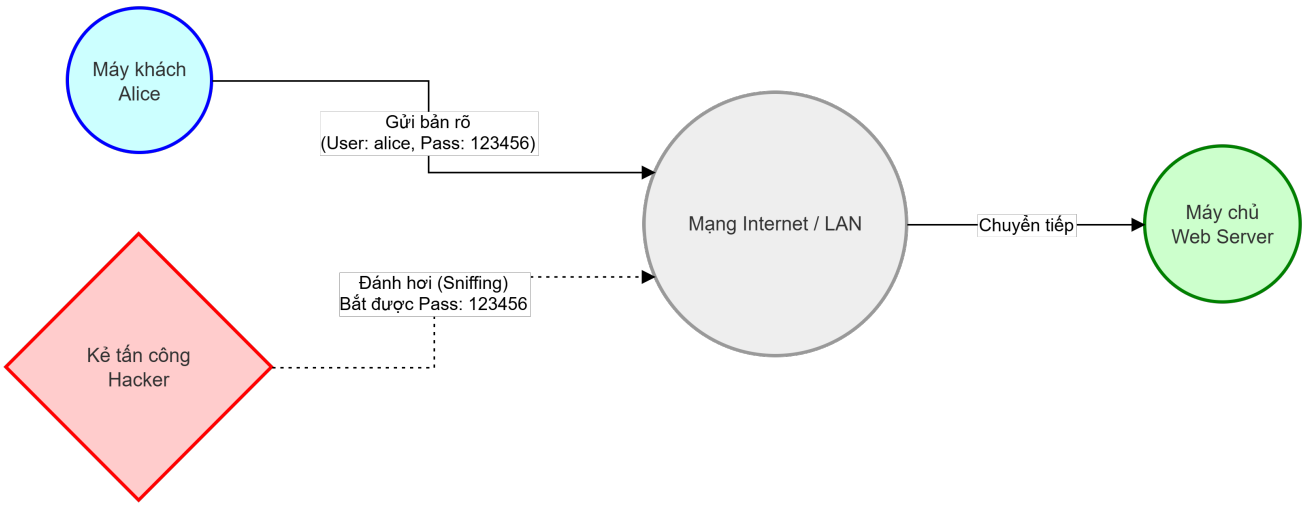
Trước những thực trạng đó, yêu cầu về một giao thức xác thực không gửi mật khẩu qua mạng, có khả năng xác thực hai chiều (Mutual Authentication) và hỗ trợ Đăng nhập một lần (Single Sign-On - SSO) trở nên cấp thiết. Đó là tiền đề cho sự ra đời của Kerberos.

## 1.2 Lịch sử hình thành và phát triển của Kerberos

Giao thức Kerberos được thai nghén và phát triển vào thập niên 1980 tại Viện Công nghệ Massachusetts (MIT) trong khuôn khổ dự án Athena. Tên gọi "Kerberos"được lấy cảm hứng từ thần thoại Hy Lạp - Cerberus, con chó săn ba đầu hung dữ canh giữ cổng địa ngục Hades. Hình tượng ba đầu này tượng trưng cho ba thành phần cốt lõi trong kiến trúc giao thức: Máy khách (Client), Máy chủ dịch vụ (Server/Service), và Trung tâm phân phối khóa (Key Distribution Center - KDC).

Quá trình phát triển của Kerberos trải qua nhiều cột mốc quan trọng:

- **Phiên bản 1 đến 3:** Đây là các phiên bản thử nghiệm nội bộ tại MIT, được sử dụng để định hình kiến trúc và tìm ra các điểm yếu trong thiết kế mã hóa.




Hình 1: Mô phỏng tấn công nghe lén (Sniffing) trong xác thực truyền thống

- **Kerberos Phiên bản 4 (v4):** Được công bố rộng rãi vào cuối thập niên 1980 do Steve Miller và Clifford Neuman thiết kế chủ đạo. Phiên bản này chủ yếu sử dụng thuật toán mã hóa DES (Data Encryption Standard). Tuy nhiên, v4 bộc lộ một số hạn chế như phụ thuộc quá nhiều vào giao thức IP, dễ bị tấn công do DES dần trở nên lỗi thời, và giới hạn thời gian sống của vé (ticket) tối đa chỉ là 21 giờ.

- **Kerberos Phiên bản 5 (v5):** Để khắc phục các yếu điểm của v4, John Kohl và Clifford Neuman đã công bố Kerberos v5 dưới dạng RFC 1510 vào năm 1993 (sau này được cập nhật lên RFC 4120 vào năm 2005). Phiên bản 5 đánh dấu một bước nhảy vọt với những cải tiến vượt bậc:

  - *Hỗ trợ đa thuật toán mã hóa:* Không chỉ giới hạn ở DES, v5 hỗ trợ 3DES, RC4, và hiện nay là AES (Advanced Encryption Standard) 128/256-bit mạnh mẽ.
  - *Độc lập giao thức mạng:* Cho phép sử dụng các địa chỉ mạng khác ngoài IP.
  - *Cơ chế ủy quyền (Delegation):* Cho phép một dịch vụ sử dụng danh tính của người dùng để truy cập một dịch vụ khác (ví dụ: Web Server truy cập Database Server thay mặt cho người dùng).
  - *Sử dụng ASN.1:* Sử dụng Abstract Syntax Notation One để định nghĩa và mã hóa cấu trúc các bản tin, giúp tăng tính linh hoạt và tương thích đa nền tảng.

Sự trưởng thành của Kerberos v5 đã thuyết phục Microsoft tích hợp nó làm giao thức xác thực mặc định cho hệ sinh thái Active Directory (AD) kể từ Windows 2000, đưa Kerberos trở thành tiêu chuẩn vàng trong các hệ thống mạng doanh nghiệp trên toàn cầu.

## 1.3 Các khái niệm cốt lõi trong hệ thống Kerberos

Để hiểu sâu về cách thức hoạt động của Kerberos, chúng ta cần nắm vững các khái niệm và thuật ngữ nền tảng sau đây:

### 1.3.1 Principal (Thực thể)

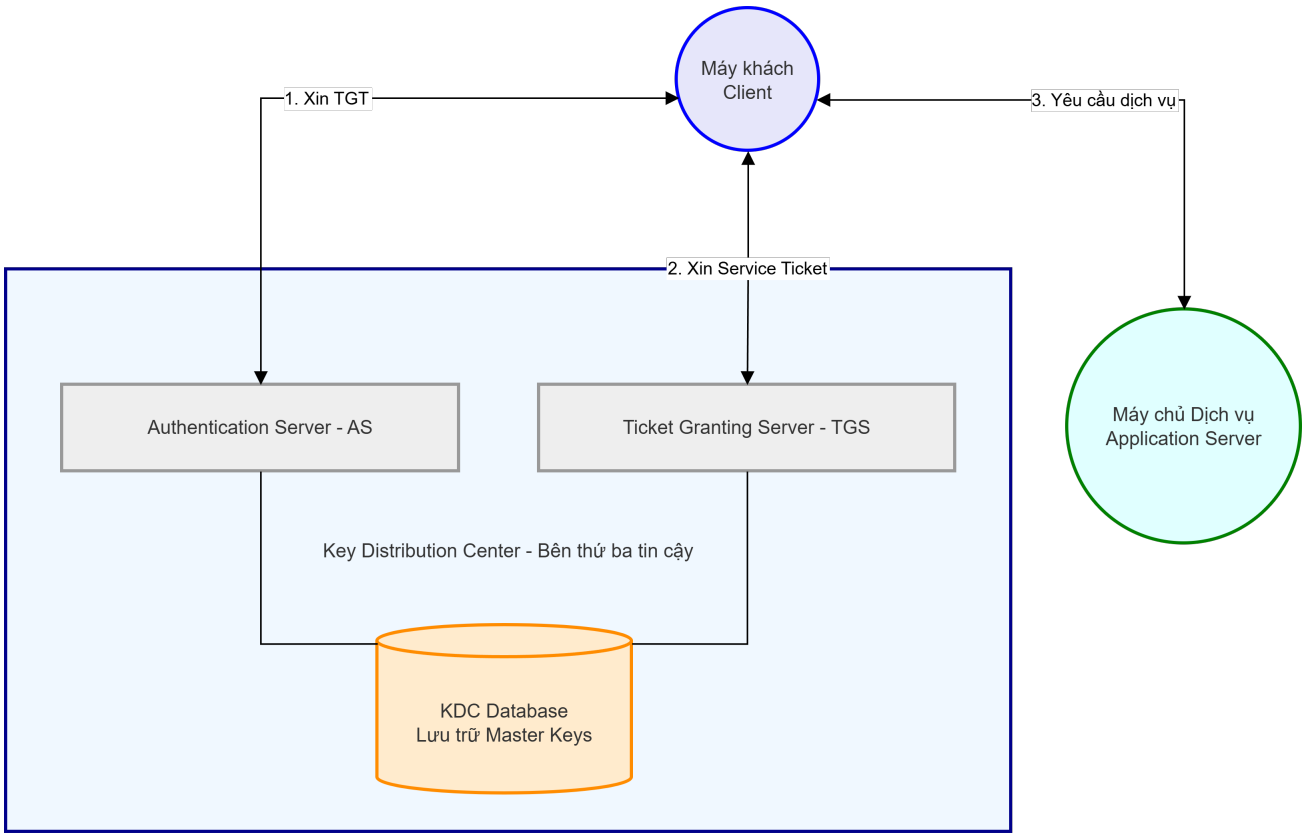
Trong Kerberos, mọi thành phần tham gia vào quá trình xác thực (bao gồm người dùng, máy tính, hoặc các dịch vụ phần mềm) đều được gọi chung là một Principal. Mỗi Principal có một định danh duy nhất. Định danh này thường có định dạng: `primary/instance@REALM`.

- **Primary:** Tên của người dùng (vd: `alice`) hoặc tên dịch vụ (vd: `HTTP`, `host`).
- **Instance:** (Tùy chọn) Cung cấp thêm thông tin về Primary. Đối với người dùng, nó có thể là vai trò (vd: `alice/admin`). Đối với dịch vụ, nó thường là Tên miền đủ điều kiện (FQDN) của máy chủ chạy dịch vụ đó (vd: `HTTP/webserver.hust.edu.vn`).
- **REALM:** Vùng quản trị của Kerberos. Tên Realm thường được viết in hoa và trùng với tên miền DNS (vd: `HUST.EDU.VN`).

### 1.3.2 Key Distribution Center (KDC - Trung tâm phân phối khóa)

KDC là trái tim của hệ thống Kerberos, đóng vai trò là "Bên thứ ba hoàn toàn tin cậy"(Trusted Third Party). Cả Client và Server đều không trực tiếp tin tưởng nhau, chúng chỉ tin tưởng KDC. KDC lưu trữ một cơ sở dữ liệu chứa Khóa bí mật (Secret Key) của tất cả các Principal trong Realm. Về mặt logic, KDC được chia thành hai tiến trình/dịch vụ riêng biệt:

1. **Authentication Server (AS):** Dịch vụ xác thực ban đầu. AS chịu trách nhiệm kiểm tra thông tin đăng nhập của người dùng và cấp phát vé TGT.
2. **Ticket Granting Server (TGS):** Dịch vụ cấp vé dịch vụ. Dựa trên TGT mà Client cung cấp, TGS sẽ kiểm tra và cấp các Service Ticket để Client truy cập vào các máy chủ ứng dụng cụ thể.




Hình 2: Kiến trúc tổng quát của hệ thống Kerberos

### 1.3.3 Ticket (Vé xác thực)

Vé là một cấu trúc dữ liệu kỹ thuật số được KDC cấp cho Client để chứng minh danh tính với một máy chủ khác một cách an toàn. Có hai loại vé chính:

- **Ticket Granting Ticket (TGT):** Được ví như "hộ chiếu"của người dùng. KDC (cụ thể là AS) cấp TGT cho Client sau lần đăng nhập thành công đầu tiên. TGT có thời hạn sử dụng (thường là 10-24 giờ) và được mã hóa bằng khóa bí mật của chính KDC. Client mang TGT này đi xin các vé dịch vụ khác mà không cần phải nhập lại mật khẩu.
- **Service Ticket (ST):** Còn gọi là TGS Ticket. Đây là "vé vào cửa"cho một dịch vụ cụ thể. Khi Client muốn truy cập File Server hay Web Server, nó dùng TGT để xin TGS cấp một ST. ST này được mã hóa bằng khóa bí mật của Server đích.

### 1.3.4 Session Key (Khóa phiên) và Authenticator (Bộ xác thực)

- **Session Key:** Là một khóa mã hóa đối xứng dùng một lần, được KDC tạo ra ngẫu nhiên cho một phiên làm việc cụ thể giữa hai bên (ví dụ: giữa Client và TGS, hoặc giữa Client và Server). Khóa phiên đảm bảo tính bí mật của dữ liệu trao đổi trong suốt phiên làm việc đó.
- **Authenticator:** Là một mẩu thông tin do Client tự tạo ra để chống lại tấn công phát lại (Replay Attack). Nó chứa Tên của Client và **Dấu thời gian (Timestamp)** hiện tại của

máy Client, sau đó được mã hóa bằng Session Key. Khi Server nhận được Authenticator, nó sẽ giải mã và kiểm tra Timestamp. Nếu Timestamp chênh lệch quá nhiều so với thời gian của Server (thường mặc định cho phép lệch tối đa 5 phút), yêu cầu sẽ bị từ chối ngay lập tức. Nhờ Timestamp này, kẻ tấn công không thể dùng lại một gói tin cũ bắt được từ hôm qua để lừa hệ thống.

# 2 CƠ SỞ LÝ THUYẾT MẬT MÃ TRONG KERBEROS

Khác với nhiều hệ thống sử dụng hạ tầng khóa công khai (PKI) với mật mã khóa bất đối xứng (như thuật toán RSA), giao thức Kerberos được xây dựng hoàn toàn dựa trên nền tảng của mật mã khóa đối xứng (Symmetric-key cryptography). Quyết định thiết kế này nhằm tối ưu hóa tốc độ xử lý xác thực cho hàng ngàn yêu cầu cùng lúc trong mạng doanh nghiệp.

## 2.1 Mật mã khóa đối xứng (Symmetric-Key Cryptography)

Mật mã khóa đối xứng là phương pháp mã hóa sử dụng chung một khóa bí mật (Secret Key - $K$) cho cả quá trình mã hóa (Encryption) và giải mã (Decryption).

Cho bản rõ $M$ (Plaintext) và bản mã $C$ (Ciphertext), hàm mã hóa $E$ và hàm giải mã $D$ được biểu diễn như sau:

$$C = E_K(M)$$
$$M = D_K(C) = D_K(E_K(M))$$

Trong hệ thống Kerberos, KDC và mỗi Principal (Client/Server) chia sẻ trước một khóa bí mật dài hạn (Long-term key). Đối với người dùng, khóa này được dẫn xuất trực tiếp từ việc băm (hash) mật khẩu của họ. Điều này đảm bảo rằng:

- KDC có thể tạo ra các bản tin mà chỉ có Principal hợp pháp mới có thể giải mã.
- Principal có thể chứng minh danh tính của mình với KDC thông qua khả năng giải mã thành công bản tin.

### 2.1.1 Các thuật toán mã hóa được Kerberos sử dụng

Lịch sử Kerberos gắn liền với sự phát triển của các chuẩn mã hóa đối xứng:

- **DES (Data Encryption Standard):** Sử dụng trong Kerberos v4 và các bản v5 thời kỳ đầu. DES mã hóa khối dữ liệu 64-bit với khóa 56-bit. Hiện nay DES đã bị loại bỏ hoàn toàn do dễ dàng bị phá vỡ bằng phương pháp vét cạn (Brute-force).
- **RC4 (Rivest Cipher 4):** Microsoft từng sử dụng RC4-HMAC làm chuẩn trong Active Directory. Tuy RC4 nhanh, nhưng nó bộc lộ nhiều điểm yếu về rò rỉ thông tin thống kê.
- **AES (Advanced Encryption Standard):** RFC 3962 đã định nghĩa việc tích hợp AES vào Kerberos v5. AES mã hóa theo khối 128-bit với các kích thước khóa 128, 192 hoặc 256-bit. Hiện tại, `AES256-CTS-HMAC-SHA1-96` là bộ thuật toán được khuyến nghị cao nhất nhờ độ an toàn tuyệt đối trước các máy tính hiện đại.

## 2.2 Khóa dài hạn (Long-term Key) và Khóa phiên (Session Key)

Hệ thống Kerberos phân biệt rất rõ ràng giữa hai loại khóa:

1. **Khóa dài hạn (Master Key):** Là khóa tồn tại cố định. Khóa của User được tạo từ mật khẩu; khóa của Service được lưu trữ trong tệp `keytab`; khóa của chính KDC (KRBTGT) được lưu sâu trong cơ sở dữ liệu. Nhược điểm của khóa dài hạn là nếu sử dụng quá nhiều lần, kẻ tấn công có thể thu thập đủ lượng bản mã để thực hiện phân tích mật mã học (Cryptanalysis).

2. **Khóa phiên (Session Key):** Để giải quyết nguy cơ trên, KDC tạo ra Session Key. Đây là những khóa ngẫu nhiên, chỉ có giá trị trong một khoảng thời gian ngắn (Ví dụ: 1 phiên đăng nhập, 1 lần kết nối đến Server). Các khóa dài hạn thực chất chỉ được dùng để "bọc"(mã hóa) và phân phối an toàn các Session Key này qua mạng. Mọi dữ liệu ứng dụng thực tế sau đó đều được mã hóa bằng Session Key.

## 2.3 Bảo vệ tính toàn vẹn (Integrity) bằng MAC và HMAC

Bên cạnh tính bí mật (Confidentiality), Kerberos phải đảm bảo tính toàn vẹn (Integrity) — tức là gói tin không bị kẻ xấu can thiệp sửa đổi trên đường truyền.

Kerberos sử dụng Mã xác thực thông điệp (Message Authentication Code - MAC). Cụ thể, các phiên bản Kerberos hiện đại sử dụng **HMAC** (Hash-based Message Authentication Code). HMAC kết hợp một hàm băm mật mã học (như MD5, SHA-1 hoặc SHA-256) với một khóa bí mật ($K$).

Cấu trúc toán học của HMAC được định nghĩa theo RFC 2104:

$$\text{HMAC}(K, M) = H((K \oplus opad) \parallel H((K \oplus ipad) \parallel M))$$

Trong đó:

- $H$ là hàm băm mật mã học (ví dụ: SHA-1).
- $M$ là thông điệp cần bảo vệ.
- $K$ là khóa bí mật đã được đệm (padding) cho đủ kích thước khối.
- $\oplus$ là phép toán XOR (Exclusive OR).
- $\parallel$ là phép nối chuỗi (Concatenation).
- *ipad* (inner pad) và *opad* (outer pad) là các hằng số đệm cố định.

Bất kỳ sự thay đổi dù là 1 bit nhỏ nhất trong thông điệp $M$ trên đường truyền cũng sẽ khiến hệ thống sinh ra một giá trị HMAC hoàn toàn khác. KDC hoặc Server khi nhận được gói tin sẽ tính toán lại HMAC; nếu không khớp với HMAC đính kèm, gói tin sẽ bị hủy bỏ ngay lập tức.

### 2.4 Nền tảng giao thức Needham-Schroeder

Về mặt học thuật, Kerberos chính là một bản thiết kế mở rộng dựa trên nguyên lý của **Giao thức khóa đối xứng Needham-Schroeder** do Roger Needham và Michael Schroeder đề xuất năm 1978. Việc hiểu giao thức này là tiền đề cốt lõi để phân tích Kerberos.

Giả sử hai thực thể $A$ (Client) và $B$ (Server) muốn thiết lập một phiên giao tiếp an toàn, nhưng họ chỉ có chung khóa bí mật với một máy chủ trung gian $S$ (đóng vai trò như KDC). Lần lượt $K_{AS}$ là khóa giữa $A$ và $S$; $K_{BS}$ là khóa giữa $B$ và $S$.

Các bước toán học của giao thức Needham-Schroeder diễn ra như sau:

1. $A \rightarrow S : A, B, N_A$
   $A$ gửi yêu cầu cho $S$ muốn nói chuyện với $B$, kèm theo một số ngẫu nhiên chỉ dùng một lần (Nonce - $N_A$) để chống phát lại.

2. $S \rightarrow A : \{N_A, K_{AB}, B, \{K_{AB}, A\}_{K_{BS}}\}_{K_{AS}}$
   $S$ tạo khóa phiên $K_{AB}$. $S$ mã hóa một gói tin gửi cho $A$ (bằng $K_{AS}$), bên trong chứa $N_A$, khóa $K_{AB}$, danh tính $B$, và một "Vé"dành cho $B$ được mã hóa bằng $K_{BS}$ (chính là nguyên thủy của Ticket trong Kerberos).

3. $A \rightarrow B : \{K_{AB}, A\}_{K_{BS}}$
   $A$ giải mã bản tin từ $S$, lấy được $K_{AB}$, kiểm tra xem $N_A$ có khớp không. Sau đó $A$ chuyển tiếp "Vé"nguyên vẹn cho $B$.

4. $B \rightarrow A : \{N_B\}_{K_{AB}}$
   $B$ nhận vé, dùng $K_{BS}$ giải mã lấy được $K_{AB}$. Để xác minh ngược lại với $A$, $B$ gửi một Nonce $N_B$ đã được mã hóa bằng khóa phiên $K_{AB}$ vừa nhận.

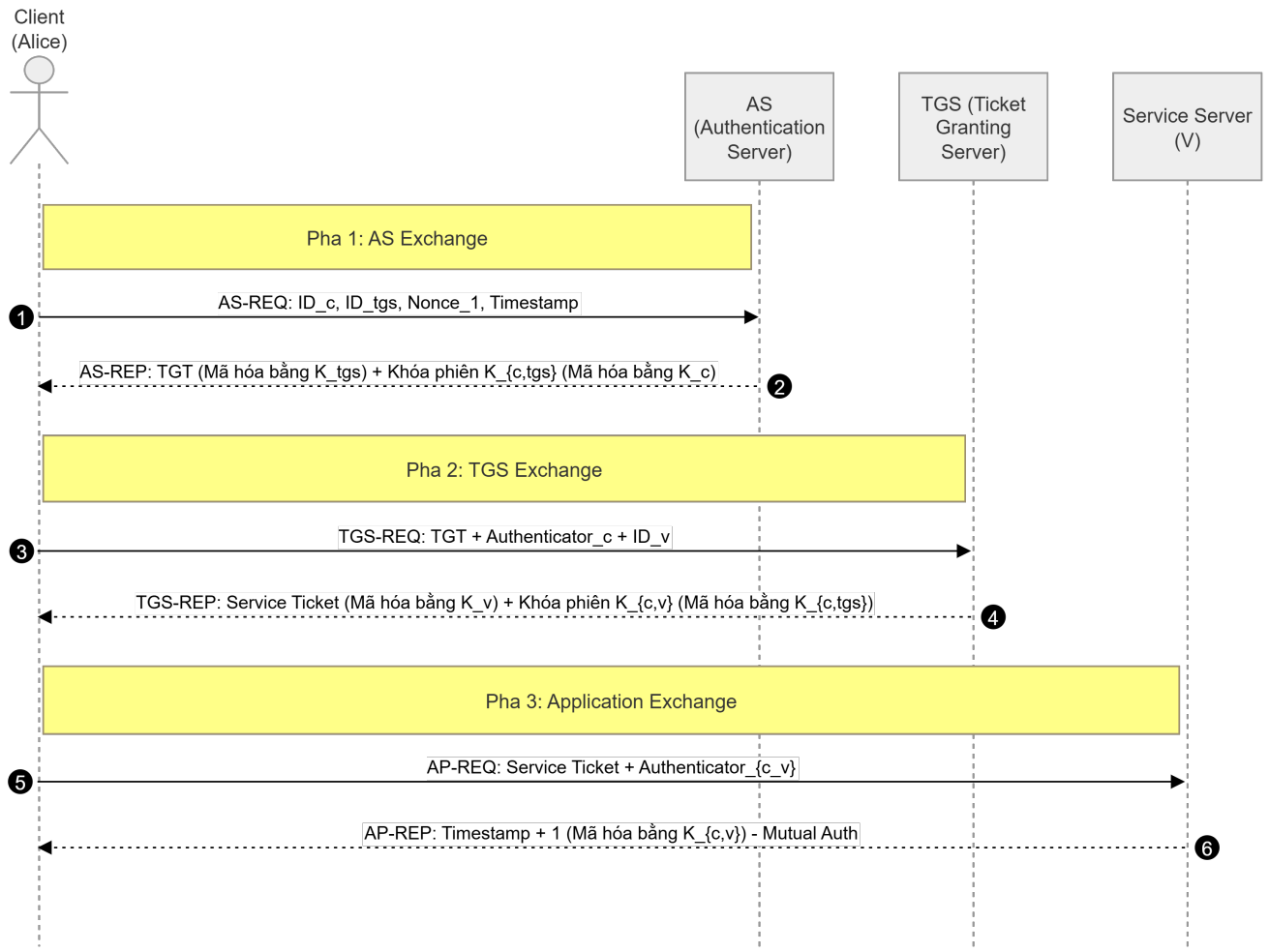
5. $A \rightarrow B : \{N_B - 1\}_{K_{AB}}$
   $A$ giải mã, lấy $N_B$ trừ đi 1 và mã hóa gửi lại cho $B$. $B$ kiểm tra, nếu đúng thì xác minh hai chiều (Mutual Authentication) thành công.

Kerberos đã kế thừa kiến trúc này, thay thế hệ thống Nonce phức tạp bằng Dấu thời gian (Timestamp) (thông qua khái niệm Authenticator) để giảm bớt số lượng bản tin phải trao đổi, đồng thời tách máy chủ trung gian $S$ thành 2 thành phần logic (AS và TGS) để gia tăng tính bảo mật và cấp quyền (Authorization).

## 3 KIẾN TRÚC VÀ CƠ CHẾ HOẠT ĐỘNG CHI TIẾT

Cơ chế hoạt động của Kerberos thường được mô tả thông qua một kịch bản gồm 3 pha giao tiếp (Exchange) với tổng cộng 6 bước bản tin (Message) giữa 3 thực thể: Client (C), Ticket Granting Server (TGS), và Service Server (V).

Giả định ban đầu: Client $C$ muốn truy cập vào máy chủ dịch vụ $V$. Client đã được đăng ký trong cơ sở dữ liệu của KDC và có khóa bí mật $K_c$ (tạo từ mật khẩu). KDC (cụ thể là AS và TGS) có khóa bí mật riêng là $K_{tgs}$. Dịch vụ $V$ có khóa bí mật $K_v$ chia sẻ với KDC.




Hình 3: Sơ đồ tổng quan 6 bước xác thực trong giao thức Kerberos

## 3.1 Pha 1: Dịch vụ xác thực ban đầu (Authentication Service Exchange)

Mục đích của pha này là Client chứng minh danh tính với KDC để lấy được "Hộ chiếu"TGT (Ticket Granting Ticket), từ đó không cần phải nhập lại mật khẩu cho các lần truy cập dịch vụ sau.

### 3.1.1 Bước 1: AS-REQ (Authentication Service Request)

Client khởi tạo quá trình đăng nhập bằng cách gửi một bản tin yêu cầu tới máy chủ xác thực AS. Điểm đặc biệt quan trọng: **Bản tin này hoàn toàn là Plaintext (bản rõ) và KHÔNG chứa mật khẩu hay hàm băm của mật khẩu.**

Cấu trúc bản tin AS-REQ bao gồm:

| Trường dữ liệu | Ý nghĩa và Nội dung |
|---|---|
| $ID_c$ | Định danh của Client (VD: username). |
| $ID_{tgs}$ | Định danh của máy chủ TGS mà Client muốn xin vé TGT. |
| $TS_1$ | Dấu thời gian (Timestamp) của Client tại thời điểm gửi để chống Replay Attack. |
| $Lifetime_1$ | Thời gian sống dự kiến của vé TGT mà Client mong muốn. |
| $Nonce_1$ | Một số ngẫu nhiên dùng một lần do Client sinh ra, dùng để ghép nối request và reply. |

Bảng 1: Cấu trúc bản tin AS-REQ

### 3.1.2 Bước 2: AS-REP (Authentication Service Reply)

Khi AS nhận được AS-REQ, nó sẽ tra cứu $ID_c$ trong cơ sở dữ liệu. Nếu tìm thấy, AS sẽ tạo ra một Khóa phiên (Session Key) ký hiệu là $K_{c,tgs}$ để dùng chung giữa Client và TGS sau này. Sau đó, AS trả về bản tin AS-REP gồm 2 phần riêng biệt:

**Phần 1: Dành cho TGS (Chính là vé TGT)** AS tạo ra TGT và mã hóa nó bằng khóa bí mật của TGS ($K_{tgs}$). Chỉ có KDC mới có thể đọc được phần này. Client sẽ lưu trữ nó ở dạng chuỗi byte vô nghĩa.

$$TGT = E_{K_{tgs}}[ID_c \parallel C_Address \parallel ID_{tgs} \parallel TS_2 \parallel Lifetime_2 \parallel K_{c,tgs}]$$

**Phần 2: Dành cho Client** AS đóng gói Khóa phiên và các thông tin liên quan, mã hóa tất cả bằng Khóa bí mật của Client ($K_c$).

$$Client_Portion = E_{K_c}[K_{c,tgs} \parallel ID_{tgs} \parallel TS_2 \parallel Lifetime_2 \parallel Nonce_1]$$

Khi nhận được AS-REP, hệ điều hành của Client sẽ yêu cầu người dùng nhập mật khẩu. Mật khẩu này được chạy qua hàm băm để tạo ra khóa $K_c$ dùng để giải mã *Client_Portion*. Nếu giải mã thành công (và $Nonce_1$ khớp), hệ thống xác nhận người dùng hợp lệ và trích xuất thành công Khóa phiên $K_{c,tgs}$.

## 3.2 Pha 2: Dịch vụ cấp vé (Ticket Granting Service Exchange)

Lúc này, Client đã có TGT và Khóa phiên $K_{c,tgs}$. Client tiến hành xin "Vé vào cửa"(Service Ticket) để truy cập máy chủ $V$.

### 3.2.1 Bước 3: TGS-REQ (Ticket Granting Service Request)

Bản tin này bao gồm vé TGT (vẫn đang bị bọc bởi khóa của TGS) và một Authenticator do Client tự tạo để chứng minh mình là chủ sở hữu hợp pháp của TGT.

| Thành phần | Chi tiết mã hóa và Dữ liệu |
|---|---|
| $ID_v$ | Định danh của Service Server $V$ muốn truy cập (Plaintext). |
| $TGT$ | Vé TGT lấy từ Bước 2. ($E_{K_{tgs}}[...]$) |
| $Authenticator_c$ | Được tạo bằng cách lấy $ID_c \parallel C_Address \parallel TS_3$. Sau đó mã hóa toàn bộ bằng Khóa phiên $K_{c,tgs}$. ($E_{K_{c,tgs}}[ID_c \parallel C_Address \parallel TS_3]$) |

Bảng 2: Cấu trúc bản tin TGS-REQ

### 3.2.2 Bước 4: TGS-REP (Ticket Granting Service Reply)

TGS nhận bản tin. Quá trình xử lý của TGS diễn ra hết sức chặt chẽ: 1. Dùng khóa bí mật $K_{tgs}$ của mình để giải mã TGT. 2. Trích xuất $ID_c$ và đặc biệt là Khóa phiên $K_{c,tgs}$ nằm trong TGT. 3. Dùng chính $K_{c,tgs}$ vừa lấy được để giải mã $Authenticator_c$. 4. Kiểm tra xem $ID_c$ trong Authenticator có khớp với $ID_c$ trong TGT hay không, và $TS_3$ có hợp lệ (không quá cũ) hay không.

Nếu mọi thứ hoàn hảo, TGS tạo ra một Khóa phiên dịch vụ (Service Session Key - $K_{c,v}$) dùng cho giao tiếp giữa Client và Server $V$. TGS phản hồi bằng AS-REP gồm 2 phần:

**Phần 1: Dành cho Server V (Service Ticket - ST)**

$$ST = E_{K_v}[ID_c \parallel C_Address \parallel ID_v \parallel TS_4 \parallel Lifetime_4 \parallel K_{c,v}]$$

**Phần 2: Dành cho Client**

$$Client_Portion = E_{K_{c,tgs}}[K_{c,v} \parallel ID_v \parallel TS_4 \parallel Nonce_2]$$

Client dùng khóa $K_{c,tgs}$ đang giữ trong RAM để giải mã, thu được $K_{c,v}$.

## 3.3 Pha 3: Trao đổi giữa Client và Server (Client/Server Exchange)

Client giờ đã nắm trong tay Service Ticket và khóa $K_{c,v}$. Pha này thực hiện Mutual Authentication (Xác thực 2 chiều).

### 3.3.1 Bước 5: AP-REQ (Application Request)

Client gửi yêu cầu truy cập đến Server $V$. Bản tin bao gồm:

- $ST$: Vé dịch vụ (Mã hóa bằng $K_v$).
- $Authenticator_{c_v}$: Authenticator mới ($E_{K_{c,v}}[ID_c \parallel C_Address \parallel TS_5]$).

Server $V$ nhận được, dùng khóa bí mật $K_v$ của mình (được lưu trong file keytab cục bộ) để giải mã $ST$. Từ $ST$, Server lấy được $K_{c,v}$ và dùng nó giải mã $Authenticator_{c_v}$. Nếu $ID_c$ khớp và Timestamp $TS_5$ hợp lệ, Server V tin tưởng Client đã được KDC cấp phép.

### 3.3.2 Bước 6: AP-REP (Application Reply - Tùy chọn nhưng quan trọng)

Để Client có thể tin tưởng ngược lại rằng Server V là máy chủ thật (không phải giả mạo), Server V trích xuất Timestamp $TS_5$ từ Authenticator của Client, cộng thêm 1 (hoặc thực hiện một phép biến đổi nhỏ), mã hóa lại bằng $K_{c,v}$ và gửi về cho Client:

$$AP - REP = E_{K_{c,v}}[TS_5 + 1]$$

Client giải mã, thấy Timestamp khớp với những gì mình vừa gửi. Lúc này, cả hai bên đã hoàn toàn tin tưởng nhau và bắt đầu trao đổi dữ liệu an toàn được mã hóa bằng $K_{c,v}$.

## 3.4 Xác thực liên miền (Cross-Realm Authentication)

Trong các tập đoàn lớn, hạ tầng mạng thường được chia thành nhiều Vùng (Realm) khác nhau (ví dụ: `HANOI.CORP.VN` và `HCM.CORP.VN`). Khi một Client ở vùng Hà Nội muốn truy cập Server ở vùng HCM, Kerberos xử lý thông qua cơ chế Cross-Realm.

Để làm được điều này, Quản trị viên phải thiết lập sự tin tưởng (Trust Relationship) giữa 2 KDC. Hai KDC sẽ chia sẻ chung một Khóa liên vùng (Inter-Realm Key).

1. Client xin TGT cục bộ từ KDC Hà Nội.
2. Client dùng TGT này xin một TGT thứ hai, gọi là TGT Liên vùng (Cross-Realm TGT) từ KDC Hà Nội, trỏ tới KDC HCM.
3. Client mang Cross-Realm TGT này đến TGS của KDC HCM để xin Service Ticket cho Server.

Cơ chế này giúp Kerberos có khả năng mở rộng (Scalability) tuyệt vời trong các kiến trúc mạng phức tạp mà vẫn duy trì tính bảo mật cao độ.

# 4 PHÂN TÍCH AN TOÀN VÀ CÁC DẠNG TẤN CÔNG TRONG KERBEROS

Mặc dù Kerberos được thiết kế với nền tảng toán học và mật mã học vững chắc, không có một hệ thống nào là an toàn tuyệt đối. Trong môi trường thực tế (đặc biệt là hệ sinh thái Microsoft Active Directory), các điểm yếu không nằm ở bản thân thuật toán mã hóa (như AES) mà thường xuất phát từ cấu hình sai, quản lý định danh yếu kém, hoặc các lỗ hổng từ hệ điều hành.

## 4.1 Mô hình hóa rủi ro và các điểm yếu nội tại

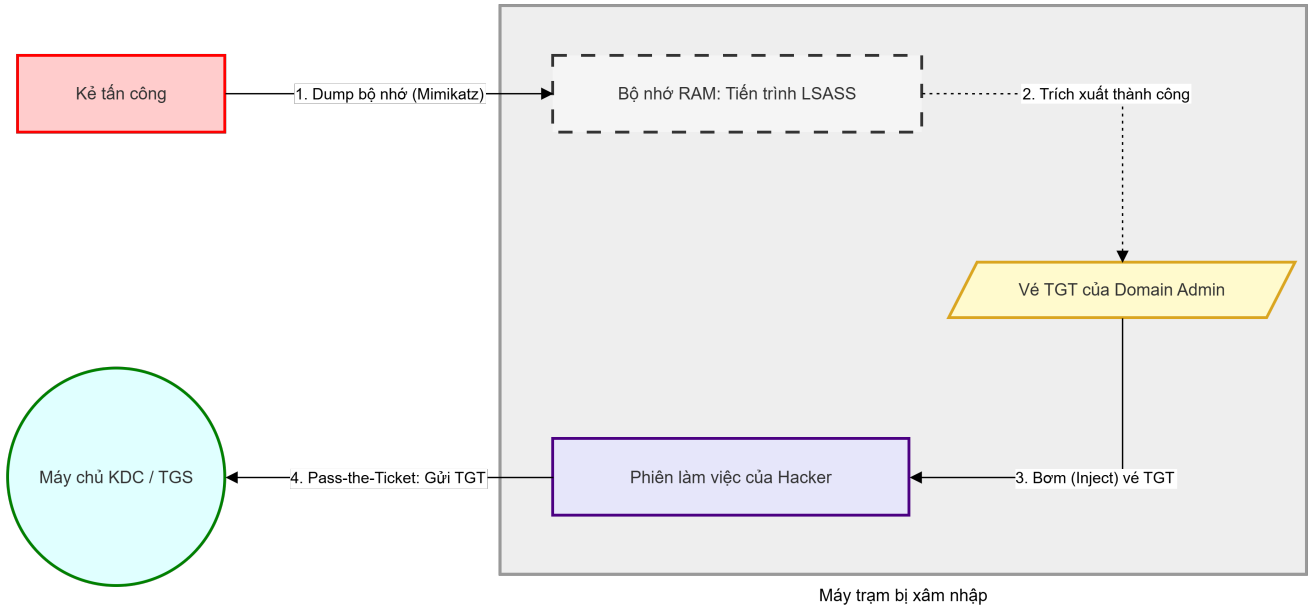
Trước khi đi sâu vào các cuộc tấn công, cần nhìn nhận những điểm yếu mang tính cấu trúc (Architectural Flaws) của Kerberos:

- **Điểm lỗi duy nhất (Single Point of Failure - SPOF):** Do Kerberos sử dụng cơ chế tập trung, KDC trở thành trái tim của toàn bộ mạng. Nếu KDC bị sập (DDoS hoặc lỗi phần cứng), toàn bộ quá trình xác thực trong mạng sẽ bị tê liệt, không ai có thể đăng nhập. Giải pháp là phải triển khai các KDC dự phòng (Secondary KDC).

- **Sự phụ thuộc nghiêm ngặt vào đồng bộ thời gian:** Cơ chế chống Replay Attack của Kerberos dựa hoàn toàn vào Dấu thời gian (Timestamp). Nếu đồng hồ giữa Client và KDC lệch nhau quá mức cho phép (mặc định của Windows là 5 phút), KDC sẽ từ chối mọi yêu cầu, dù mật khẩu hoàn toàn chính xác. Điều này đòi hỏi mạng phải có hệ thống máy chủ NTP (Network Time Protocol) cực kỳ ổn định. Kẻ tấn công có thể lợi dụng điều này bằng cách giả mạo gói tin NTP để làm sai lệch thời gian của Client, gây ra lỗi từ chối dịch vụ (DoS).
- **Nguy cơ từ việc lưu trữ khóa cục bộ:** Để giảm thiểu việc nhập mật khẩu nhiều lần, hệ điều hành (như Windows LSASS - Local Security Authority Subsystem Service) phải lưu trữ các vé TGT và Session Key trong bộ nhớ RAM. Nếu máy Client bị xâm nhập, hacker có thể trích xuất các vé này.

## 4.2 Tấn công Pass-the-Ticket (PtT)

Pass-the-Ticket là một kỹ thuật khai thác hậu xâm nhập (Post-exploitation). Trong tấn công này, hacker không cần biết mật khẩu của người dùng.




Hình 4: Cơ chế khai thác vé TGT trong bộ nhớ của tấn công Pass-the-Ticket

**Cơ chế hoạt động:**

1. Hacker xâm nhập thành công vào một máy trạm (Workstation) thông qua Malware hoặc Phishing.
2. Hacker leo thang đặc quyền lên mức Administrator hoặc SYSTEM.
3. Sử dụng các công cụ chuyên dụng như `Mimikatz` hoặc `Rubeus`, hacker kết xuất (dump) toàn bộ bộ nhớ của tiến trình LSASS.

4. Hacker tìm thấy vé TGT hoặc Service Ticket (ST) hợp lệ của một người dùng khác (có thể là Domain Admin vừa đăng nhập tạm vào máy này).

5. Hacker "bơm"(inject) vé TGT này vào phiên làm việc của chính mình. Từ đó, hacker có thể sử dụng các tài nguyên mạng dưới tư cách của người dùng bị đánh cắp cho đến khi vé hết hạn.

**Biện pháp phòng chống:** Hạn chế cấp quyền Local Admin cho người dùng thông thường. Kích hoạt tính năng *Windows Defender Credential Guard* để cô lập tiến trình LSASS vào một vùng chứa ảo hóa, ngăn chặn Mimikatz đọc bộ nhớ.

## 4.3 Tấn công Kerberoasting

Kerberoasting là một trong những kỹ thuật phổ biến và nguy hiểm nhất đối với Active Directory, vì nó có thể được thực hiện bởi bất kỳ một người dùng thông thường nào (không cần đặc quyền Admin) và không hề tạo ra các log cảnh báo đáng ngờ.

**Cơ chế hoạt động:** Nhắc lại *Bước 4* trong cơ chế Kerberos: Khi Client yêu cầu truy cập một dịch vụ, TGS sẽ trả về một Service Ticket (ST) được mã hóa bằng khóa bí mật (mã băm mật khẩu) của **Tài khoản chạy dịch vụ đó (Service Account)**.

1. Hacker đăng nhập vào mạng bằng một tài khoản nhân viên bình thường.

2. Hacker quét (query) LDAP để tìm các tài khoản dịch vụ có gắn Service Principal Name (SPN).

3. Hacker gửi yêu cầu TGS-REQ xin vé để truy cập vào các dịch vụ đó. KDC, làm đúng nhiệm vụ của mình, trả về TGS-REP chứa Service Ticket được mã hóa bằng mã băm của tài khoản dịch vụ.

4. Hacker mang các Service Ticket này về máy tính cá nhân.

5. Hacker sử dụng các phần mềm phá pass (như `Hashcat` hoặc `John the Ripper`) để thực hiện tấn công Brute-force/Dictionary offline. Vì làm việc offline, hệ thống không hề biết và không có cơ chế khóa tài khoản (Account Lockout) nào được kích hoạt.

Nếu quản trị viên đặt mật khẩu cho tài khoản dịch vụ quá yếu, hacker sẽ nhanh chóng dò ra mật khẩu rõ.

**Biện pháp phòng chống:** Đảm bảo mọi tài khoản Service (có SPN) phải sử dụng mật khẩu cực kỳ phức tạp (trên 25 ký tự, sinh ngẫu nhiên). Ép buộc hệ thống sử dụng thuật toán mã hóa AES-256 thay vì RC4 cũ kỹ, vì giải mã AES offline tốn rất nhiều tài nguyên máy tính.

## 4.4 Tấn công Golden Ticket và Silver Ticket

Đây là đỉnh cao của việc thỏa hiệp hệ thống Kerberos, cho phép hacker thiết lập quyền cai trị vĩnh viễn (Persistence) trong mạng.

### 4.4.1 Golden Ticket (Vé Vàng)

Vé vàng xảy ra khi hacker chiếm quyền kiểm soát được máy chủ Domain Controller và đánh cắp được mã băm mật khẩu của tài khoản **KRBTGT**. Tài khoản KRBTGT chính là tài khoản đại diện cho dịch vụ KDC.

- **Nguyên lý:** Nhắc lại *Bước 2*, TGT được mã hóa bằng khóa của KDC. Nếu có khóa của KRBTGT, hacker không cần KDC nữa. Chúng có thể *tự rèn (forge)* ra các vé TGT hợp lệ.
- **Mức độ nguy hiểm:** Hacker có thể tự cấp cho mình TGT với tên bất kỳ (kể cả những user không tồn tại), gán quyền Domain Admin, và thiết lập thời hạn vé là 10 năm. Việc thay đổi mật khẩu của các Admin khác không có tác dụng. Hệ thống hoàn toàn bị kiểm soát.

### 4.4.2 Silver Ticket (Vé Bạc)

Tương tự như Vé Vàng, nhưng thay vì đánh cắp khóa của KDC, hacker đánh cắp khóa bí mật của một **Service Server** cụ thể (ví dụ: Database Server).

- **Nguyên lý:** Hacker tự tạo ra Service Ticket (TGS) hợp lệ và mã hóa bằng khóa của Server đích.
- **Mức độ nguy hiểm:** Hacker gửi thẳng Service Ticket giả mạo này tới Server (Bước 5) mà không cần giao tiếp với KDC. KDC sẽ không ghi nhận bất kỳ log xác thực nào, khiến cuộc tấn công hoàn toàn tàng hình. Tuy nhiên, phạm vi ảnh hưởng chỉ giới hạn ở máy chủ dịch vụ bị lộ khóa.

**Biện pháp phòng chống:** Bảo vệ tối đa vật lý và logic cho máy chủ Domain Controller. Thực hiện quy trình thay đổi mật khẩu tài khoản KRBTGT định kỳ hai lần liên tiếp (do Windows lưu trữ mật khẩu hiện tại và 1 mật khẩu quá khứ).

## 4.5 Đánh giá lại Tấn công phát lại (Replay Attack)

Kerberos chống lại Replay Attack thông qua cơ chế Authenticator và Dấu thời gian (Timestamp). Khi Server nhận được gói tin ở Bước 5, nó lưu Timestamp vào một bộ nhớ đệm (Replay Cache). Nếu hacker bắt được gói tin và gửi lại vài giây sau đó, Server sẽ tra cứu trong Replay Cache. Mặc dù Timestamp vẫn nằm trong khoảng cho phép (5 phút), nhưng do nó đã tồn tại trong Cache, Server sẽ phát hiện đây là gói tin cũ và lập tức đánh rớt. Sự kết hợp giữa giới hạn thời gian 5 phút và Replay Cache tạo nên bức tường thành vững chắc chống lại các nỗ lực nghe lén và phát lại.

# 5 CHƯƠNG TRÌNH THỬ NGHIỆM VÀ MÔ PHỎNG

Thay vì sử dụng các thư viện xác thực cấp cao có sẵn (như JAAS GSS-API của Java hay thư viện MIT Kerberos) vốn ẩn đi các chi tiết kỹ thuật phức tạp bên dưới, nhóm chúng em đã quyết định tự xây dựng một chương trình mô phỏng giao thức Kerberos bằng ngôn ngữ C++.

Mục tiêu của chương trình này không phải là tạo ra một hệ thống mã hóa chuẩn công nghiệp, mà là để minh họa trực quan và chính xác **luồng logic trao đổi bản tin (Message Flow)** qua 3 bước cốt lõi: AS Exchange, TGS Exchange, và Application Exchange.

## 5.1 Môi trường và Công cụ thử nghiệm

Chương trình được phát triển và thử nghiệm trên môi trường cơ bản sau:

- **Ngôn ngữ lập trình:** C++ (Tiêu chuẩn C++17).
- **Công cụ quản lý dự án (Build Tool):** CMake (phiên bản $\geq 3.16$).
- **Môi trường biên dịch:** Trình biên dịch hỗ trợ C++17 (MSVC/GCC/Clang) trên hệ điều hành Windows (chạy qua giao diện PowerShell).

## 5.2 Thiết kế và Cấu trúc mã nguồn

Hệ thống mô phỏng được thiết kế bám sát vào kiến trúc lý thuyết của Kerberos, bao gồm các thành phần (Class/Module) được phân tách rõ ràng:

- **KDC Database (`kdc_database.cpp`):** Mô phỏng cơ sở dữ liệu của KDC, lưu trữ danh sách các Principal (User, TGS, Service) và khóa bí mật dài hạn của họ.
- **Authentication Server (`as_server.cpp`):** Xử lý bản tin AS-REQ, xác thực người dùng và cấp vé Ticket Granting Ticket (TGT) cùng Khóa phiên $K_{c,tgs}$.
- **Ticket Granting Server (`tgs_server.cpp`):** Xử lý bản tin TGS-REQ, kiểm tra TGT và Authenticator, sau đó cấp Service Ticket và Khóa phiên $K_{c,v}$.
- **Service Server (`service_server.cpp`):** Máy chủ đích, nhận AP-REQ từ Client, kiểm tra Service Ticket, xác thực Authenticator (có đối chiếu Timestamp để chống Replay Attack) và phản hồi AP-REP (Mutual Authentication).
- **Kerberos Client (`kerberos_client.cpp`):** Kịch bản hóa toàn bộ 3 pha giao tiếp từ phía người dùng.
- **Crypto Utils (`crypto_utils.cpp`):** Để phục vụ mục đích theo dõi logic bản tin dễ dàng trong học tập, cơ chế mã hóa phức tạp (AES) được thay thế bằng phép toán XOR Stream và các hàm Băm (Hash) cơ bản.

## 5.3 Cảnh báo học thuật và Phạm vi mô phỏng

**Giới hạn của mô phỏng:** Như đã đề cập, nhằm mục đích sư phạm và đơn giản hóa việc theo dõi dòng chảy dữ liệu (Data flow), chương trình sử dụng mã hóa XOR. Tuyệt đối không sử dụng module mã hóa này cho các ứng dụng thực tế. Chương trình cũng lược bỏ các yếu tố như Ticket Cache, PAC (Privilege Attribute Certificate) hay chính sách Domain phức tạp của Active Directory.

**Điểm bám sát lý thuyết:** Dù tối giản hóa mật mã, chương trình **đáp ứng 100% logic giao thức Kerberos**:

- Hệ thống KDC được chia tách rõ ràng thành AS và TGS.
- Gói tin được bảo vệ thông qua khái niệm Ticket (TGT, ST).

- Có sự hiện diện của Session Key cấp theo từng phiên.
- Sử dụng Authenticator kết hợp Timestamp để chống phát lại và thực hiện xác thực hai chiều (Mutual Authentication).

## 5.4 Phân tích Kịch bản khởi chạy và Cấu trúc hệ thống (main.cpp)

Tệp main.cpp đóng vai trò là điểm entry-point, nơi hệ thống khởi tạo cơ sở dữ liệu KDC giả lập và thiết lập các khóa bí mật dài hạn (Long-term keys).

```cpp
int main() {
    using namespace demo;
    std::cout << "===== Kerberos Protocol Demo (Educational) =====\n";

    // 1. Khoi tao KDC Database va dang ky Principal
    KdcDatabase db;
    db.add_client("alice", "alice_password_123");

    // 2. Thiet lap Khoa bi mat (Master Keys)
    const std::string tgs_secret = "TGS_SECRET_KEY";
    const std::string mail_service_secret = "MAIL_SERVICE_SECRET";

    // 3. Khoi tao cac Server Component
    AuthenticationServer as_server(db, tgs_secret, 300);
    TicketGrantingServer tgs_server(tgs_secret, { {"mail-service",
        mail_service_secret} }, 300);
    ServiceServer mail_service(mail_service_secret);
    // ...
```

Listing 1: Khởi tạo KDC và các Service

Trong đoạn mã trên, có thể thấy rõ kiến trúc phân mảnh của Kerberos. Cơ sở dữ liệu KdcDatabase lưu trữ thông tin người dùng alice. Máy chủ AS được khởi tạo với khóa của KDC (tgs_secret). Tương tự, máy chủ TGS nắm giữ danh sách các dịch vụ hợp lệ (trong trường hợp này là mail-service) cùng với khóa bí mật chia sẻ tương ứng. Số 300 đại diện cho thời gian sống của vé (Ticket Lifetime).

Hệ thống cung cấp hai kịch bản (Scenarios) để chứng minh tính đúng đắn của logic:

- **Scenario 1:** Người dùng alice cung cấp đúng mật khẩu. Kịch bản mong đợi là hoàn thành toàn bộ 3 pha và xác thực thành công.
- **Scenario 2:** Một kẻ tấn công cố gắng đóng giả alice nhưng sử dụng mật khẩu sai. Kịch bản mong đợi là hệ thống từ chối ngay tại pha AS.

## 5.5 Phân tích Luồng xác thực từ góc nhìn Client (kerberos_client.cpp)

Hàm login_and_access_service thực thi chính xác 6 bước của giao thức Kerberos. Dưới đây là phân tích chi tiết từng pha dựa trên mã nguồn.

### 5.5.1 Pha 1: AS Exchange (Xin cấp TGT)

```
    std::cout << "[Client] Sending AS-REQ...\n";
    const auto nonce = generate_nonce();
    auto as_rep = as_server.process_as_req(client_id_, password_, tgs_id_,
        nonce);

    if (!as_rep.ok) { return false; }

    // Client tu bam mat khau de tao khoa Kc
    auto client_key = hash_password(password_);
    // Dung Kc de giai ma phan tin nhan danh rieng cho Client
    auto as_plain = decrypt_text(as_rep.encrypted_part_for_client,
        client_key);
    auto as_parts = split_pipe(as_plain);

    // Kiem tra tinh toan ven va Nonce
    if (as_parts.size() != 4 || as_parts[1] != nonce) { return false; }

    auto c_tgs_session_key = as_parts[0]; // Trich xuat Khoa phien K_c,tgs
    auto encrypted_tgt = as_parts[3];     // Luu tru TGT (dang bi ma hoa)
```

Listing 2: Client thực hiện AS Exchange

Như lý thuyết đã trình bày, Client không gửi mật khẩu mà chỉ gửi thông tin `client_id` và `nonce` (AS-REQ). Khi nhận được AS-REP, quá trình quan trọng nhất diễn ra: Client sử dụng hàm `hash_password` để mô phỏng việc tạo khóa $K_c$. Nếu mật khẩu đúng, hàm `decrypt_text` sẽ giải mã thành công, cho phép Client trích xuất Khóa phiên $K_{c,tgs}$ (`c_tgs_session_key`) và lưu giữ vé TGT. Việc kiểm tra biến `nonce` giúp hệ thống phòng chống Replay Attack ngay từ pha đầu tiên.

### 5.5.2 Pha 2: TGS Exchange (Xin cấp Service Ticket)

```
    std::cout << "[Client] Sending TGS-REQ...\n";
    // 1. Tao Authenticator voi Timestamp
    Authenticator tgs_auth{client_id_, now_unix_seconds()};

    // 2. Ma hoa Authenticator bang Session Key K_c,tgs
    auto enc_tgs_auth = encrypt_text(serialize_authenticator(tgs_auth),
        c_tgs_session_key);

    // 3. Gui TGS-REQ gom TGT va Authenticator
    auto tgs_rep = tgs_server.process_tgs_req(service_id, encrypted_tgt,
        enc_tgs_auth);
    if (!tgs_rep.ok) { return false; }

    // 4. Giai ma TGS-REP de lay Khoa phien dich vu
    auto tgs_plain = decrypt_text(tgs_rep.encrypted_part_for_client,
        c_tgs_session_key);
    auto tgs_parts = split_pipe(tgs_plain);

    auto c_s_session_key = tgs_parts[0]; // Trich xuat Khoa phien K_c,v
    auto encrypted_service_ticket = tgs_parts[3];
```

Listing 3: Client thực hiện TGS Exchange

Trong đoạn mã này, khái niệm **Authenticator** được thể hiện cực kỳ rõ ràng. Bằng cách gọi `now_unix_seconds()`, Client gắn dấu thời gian ($TS$) vào cấu trúc `tgs_auth`. Việc mã hóa Authenticator bằng khóa $K_{c,tgs}$ chứng minh với TGS rằng người gửi đang thực sự sở hữu khóa phiên hợp pháp được cấp ở Pha 1. Kết thúc pha này, Client thu thập được Vé dịch vụ (`encrypted_service_ticket`) và Khóa phiên dịch vụ $K_{c,v}$ (`c_s_session_key`).

### 5.5.3 Pha 3: Application Exchange (Xác thực hai chiều)

```
    std::cout << "[Client] Sending AP-REQ to service...\n";
    // 1. Tao Authenticator MOI cho dich vu
    Authenticator service_auth{client_id_, now_unix_seconds()};
    auto enc_service_auth =
        encrypt_text(serialize_authenticator(service_auth), c_s_session_key);

    // 2. Gui AP-REQ toi Server
    auto ap_rep = service.process_ap_req(encrypted_service_ticket,
        enc_service_auth);
    if (!ap_rep.ok) { return false; }

    // 3. Giai ma ban tin AP-REP tu Server
    auto proof_plain = decrypt_text(ap_rep.encrypted_server_proof,
        c_s_session_key);

    // 4. Kiem tra Mutual Authentication (Timestamp + 1)
    if (proof_plain != std::to_string(service_auth.timestamp + 1)) {
        std::cout << "[Client] AP-REP proof invalid\n";
        return false;
    }

    std::cout << "[Client] Mutual authentication success. Service
        accessed.\n";
    return true;
```

Listing 4: Xác thực hai chiều giữa Client và Service

Đây là minh chứng lập trình sống động nhất cho cơ chế **Mutual Authentication (Xác thực hai chiều)**. Sau khi gửi AP-REQ, Client nhận về một bản tin xác minh từ Server (`encrypted_server_proof`). Lệnh `if (proof_plain != std::to_string(service_auth.timestamp + 1))` hoàn toàn khớp với công thức toán học $E_{K_{c,v}}[TS_5 + 1]$ ở Chương 3. Chỉ khi máy chủ đích giải mã thành công Authenticator, lấy được Timestamp và cộng thêm 1 rồi gửi lại, Client mới hoàn toàn tin tưởng đây là máy chủ `mail-service` hợp pháp, không phải một kẻ giả mạo (Spoofing).

## 5.6 Kết luận thử nghiệm

Chương trình mô phỏng bằng C++ đã tái hiện thành công sự tinh tế trong thiết kế của giao thức Kerberos. Dù sử dụng các phép toán mật mã đơn giản hóa, luồng dữ liệu (Data flow) chứng minh rằng: Mật khẩu không bao giờ rời khỏi Client; KDC hoàn thành xuất sắc vai trò môi giới tin

cậy (Broker); và Timestamp đóng vai trò sinh tử trong việc duy trì tính toàn vẹn của chuỗi xác thực.

# KẾT LUẬN CHUNG

Qua quá trình nghiên cứu và thực hiện bài tập lớn, nhóm đã đi sâu phân tích giao thức Kerberos từ lý thuyết mật mã nền tảng, kiến trúc 6 bước trao đổi bản tin, cho đến việc tự tay lập trình mô phỏng lại luồng giao tiếp phức tạp này.

Kerberos đã chứng minh được lý do tại sao nó là chuẩn mực xác thực trong môi trường doanh nghiệp hiện đại: An toàn, minh bạch với người dùng (SSO), và khả năng phòng chống Replay Attack hiệu quả thông qua hệ thống vé và dấu thời gian. Tuy nhiên, qua phân tích rủi ro (Pass-the-Ticket, Kerberoasting), chúng em nhận thấy không có công cụ bảo mật nào là "viên đạn bạc". Để hệ thống thực sự an toàn, giao thức phải đi kèm với việc quản trị máy chủ (AD) nghiêm ngặt, cấu hình mật khẩu phức tạp và giám sát các hành vi bất thường trên mạng.

Đây là một bài tập lớn mang lại rất nhiều kiến thức nền tảng quý báu, không chỉ rèn luyện tư duy thiết kế hệ thống phân tán mà còn rèn luyện kỹ năng lập trình an toàn thông tin. Nhóm hy vọng rằng với những nỗ lực tìm hiểu và minh họa thực tế, báo cáo đã đáp ứng được các tiêu chí khắt khe mà học phần đề ra.

# TÀI LIỆU THAM KHẢO


1. B. Clifford Neuman, Theodore Ts'o (1994). *Kerberos: An Authentication Service for Computer Networks*. IEEE Communications.

2. William Stallings, "Cryptography and Network Security", 7th Edition.

3. J. Kohl, C. Neuman (1993), RFC 1510: *The Kerberos Network Authentication Service (V5)*.

4. Microsoft Documentation: *Kerberos Authentication Overview*, [Online], https://docs.microsoft.com/en-us/windows-server/security/kerberos/kerberos-authentication-overview

5. Bài giảng "Nhập môn An toàn thông tin" – PGS. TS. Nguyễn Linh Giang.